# Toxic Comment Classification

Nguyen Thi Huyen Trang
Student ID: 20251322M
School of Information and Communications Technology
Hanoi, Vietnam
trang.nth251322m@sis.hust.edu.vn

Mai The Son
Student ID: 20252079M
School of Information and Communications Technology
Hanoi, Vietnam
son.mt252079m@sis.hust.edu.vn

Phan Duc Thinh
Student ID: 20252261M
School of Information and Communications Technology
Hanoi, Vietnam
thinh.pd252261m@sis.hust.edu.vn

Hamidova Sanubar Alisa
Student ID: 20252893M
School of Information and Communications Technology
Hanoi, Vietnam
alisa.hs252893m@sis.hust.edu.vn

Nguyen Minh Huyen
Student ID: 20252007M
School of Information and Communications Technology
Hanoi, Vietnam
huyen.nm252007m@sis.hust.edu.vn

## Abstract

Multi-label toxic comment classification involves identifying overlapping forms of abusive language in highly imbalanced text data. This study investigates three design choices on the Jigsaw Toxic Comment Classification dataset: text representation, neural architecture, and explicit label-dependency modeling. For the representation comparison, we evaluate TF-IDF and five pretrained embedding resources combined with six pooling strategies, using one-vs-rest logistic regression as the downstream classifier. We then compare recurrent and convolutional architectures using a common pretrained embedding source, and assess a label-dependency extension against a standard RoBERTa multi-label classifier. Macro ROC-AUC is the primary evaluation metric. The representation comparison uses a shared holdout partition, while the neural architecture and RoBERTa comparisons use five-fold evaluation.

For the representation comparison, FastText with concatenated max and supervised-attention pooling achieves the highest Macro ROC-AUC of 0.97956, followed by GloVe Twitter with the same pooling strategy at 0.97779, compared with 0.97541 for TF-IDF. Although this combined pooling strategy performs best within each embedding source, only two of the thirty embedding–pooling configurations outperform TF-IDF. With frozen FastText vectors, learned sequence encoders (0.9851–0.9877) outperform logistic regression (0.9715) and LightGBM (0.9800) on pooled features, while the gated recurrent models and DPCNN obtain closely matched scores. Adding explicit label dependency does not improve RoBERTa: mean Macro ROC-AUC is 0.9881 for the extended model, compared with 0.9887 for the baseline. A modular Python implementation supports reusable evaluation and interchangeable in-memory and CSV dataset access. Overall, the findings highlight the importance of evaluating representation choices jointly and validating additional model complexity against matched baselines.

## CCS Concepts

• **Computing methodologies** → **Natural language processing**; *Supervised learning by classification*; Neural networks.

## Keywords

toxic comment classification, multi-label text classification, class imbalance, label dependency, transformers, content moderation

## 1 Introduction

User-generated content is central to how people communicate on modern online platforms – comment sections, forums, and social media all rely on open participation to function. That same openness, however, exposes platforms to harassment, hate speech, and other forms of toxic language that can drive users away and, at scale, cause real harm. Manually reviewing every comment is not feasible for platforms that receive thousands or millions of posts per day, which has made automated content moderation – flagging or filtering harmful text before or shortly after it is posted – a practical necessity rather than a convenience.

The *Jigsaw Toxic Comment Classification Challenge*[1], released by the Conversation AI team (a collaboration between Jigsaw and Google), formalized this problem using a large corpus of Wikipedia talk page comments, each manually annotated for six overlapping categories of toxicity: *toxic*, *severe_toxic*, *obscene*, *threat*, *insult*, and *identity_hate*. The dataset has since become a standard benchmark for evaluating text classification methods under realistic conditions – most notably, the coexistence of multiple toxicity types within a single comment and a highly uneven distribution of these categories in practice, where severe categories such as *threat* are rare but disproportionately important to catch.

This project uses the Jigsaw dataset as a testbed to study a more general question: as text classification techniques evolve, from linear bag-of-words models to deep learning and, more recently, large pretrained language models, which of these advances actually translate into better multi-label toxic comment detection? Section 3 formalizes this question; the remainder of the report designs, implements, and empirically evaluates a set of methods intended to answer it.

Our contributions are:

- a comparison of TF-IDF with five pretrained embeddings under six pooling strategies, using a fixed logistic regression classifier;

[1] https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge

- a fold-paired comparison of pooled-feature classifiers (logistic regression and LightGBM), recurrent encoders, and a convolutional encoder built on the same frozen FastText vectors;
- a controlled ablation of a label-dependency head on RoBERTa, with a per-label analysis of the rare labels.

## 2 Related Work

**Toxic and abusive language detection.** Wulczyn et al. [21] built the Wikipedia talk page corpus that underlies the Jigsaw challenge and showed that character n-gram logistic regression can detect personal attacks at scale. Davidson et al. [4] highlighted the difficulty of separating hate speech from merely offensive language, and two surveys summarize features and models used for the task [7, 17]. On the Jigsaw data specifically, van Aken et al. [19] compared logistic regression, CNNs, Bi-GRUs with attention, and ensembles, and found that the remaining errors are dominated by rare labels, missing context, and annotation inconsistencies.

**Text classification models.** Linear models on bag-of-n-gram features remain strong baselines; NB-SVM [20] rescales these features with Naive Bayes log-count ratios. Gradient-boosted decision trees such as LightGBM [12] are a standard choice for tabular data and are often applied to TF-IDF features. Convolutional models capture local n-gram patterns with learned filters [13]; DPCNN [11] stacks residual convolution blocks with downsampling to model long-range context at low cost. Recurrent encoders read text as a sequence in both directions [18]; gated cells such as the GRU [3] mitigate vanishing gradients [1], and attention pooling [22] weighs the most informative tokens. Pretrained transformers such as BERT [5] and RoBERTa [14] now set the state of the art on most text classification benchmarks.

**Multi-label learning and label dependency.** Binary relevance, i.e., one independent classifier per label, ignores label correlations [23]. ML-GCN [2] models these correlations with a graph over labels whose edges are conditional co-occurrence probabilities. Our label-dependency layer follows the same idea, but uses a GraphSAGE-style update [8] on top of a RoBERTa text encoder.

## 3 Problem Formulation

Let $x$ be a comment and $\mathbf{y} = (y_1, \ldots, y_L) \in \{0,1\}^L$ its label vector over the $L = 6$ toxicity categories. Labels are not mutually exclusive, so a comment can have any subset of labels, including none. Given a training set $\mathcal{D} = \{(x_n, \mathbf{y}_n)\}_{n=1}^N$, the goal is to learn a scoring function $f : x \mapsto \mathbf{s} \in [0,1]^L$, where $s_l$ estimates $P(y_l = 1 \mid x)$, and a decision rule $\hat{y}_l = [s_l \geq \tau_l]$ with a threshold $\tau_l$ per label.

Two properties of the data make the problem hard: *class imbalance*, since some labels have well under 1% positives, and *label dependency*, since some labels almost always co-occur (Section 5.1). We therefore ask three research questions:

- **RQ1 (Text representation).** How do TF-IDF representations and pooled pretrained embeddings compare in terms of Macro ROC-AUC for toxic comment classification when evaluated with the same classifier and evaluation protocol?
- **RQ2 (Model architecture).** How does the choice of model architecture—pooled-feature classifiers, convolutional models, or recurrent models—affect Macro ROC-AUC when using the same pretrained embedding source and evaluation protocol?
- **RQ3 (Label dependency).** How does explicitly modeling label dependencies affect Macro ROC-AUC relative to a standard RoBERTa multi-label classifier under the same training and evaluation protocol?

## 4 Proposed Method

### 4.1 Framework Overview

All experiments run in one shared framework (Figure 1). The framework fixes every part of the pipeline except the factor that a research question studies. A difference in scores can therefore be attributed to that factor, and not to the data, the split, or the evaluation code.

**Shared pipeline.** The top row of Figure 1 shows the five stages that every model goes through.

(1) *Data access.* The training data is loaded through a dataset repository. Two back ends implement the same `load` and `save` interface: an in-memory repository for notebooks and unit tests, and a CSV repository on the file system. Switching between them requires no change in the training or evaluation code.
(2) *Preprocessing.* Every comment is cleaned once at two levels, which gives a light and a heavy version of its text (Section 5.1). The cleaned data is stored and reused, so all models read exactly the same text.
(3) *Splitting.* The cleaned data is split once into five stratified folds with a fixed seed. Every model is trained and tested on the same folds, which also allows paired, fold-by-fold comparisons between models.
(4) *Modeling.* Each model is a scoring function $f = g \circ \phi$. The representation $\phi$ turns a comment into features, and the classifier $g$ maps these features to six label scores $\mathbf{s} \in [0,1]^6$ (Section 3). All models, from logistic regression to a fine-tuned RoBERTa, expose the same scikit-learn-style interface (`fit`, `predict`, `predict_proba`) and are created by name from a model registry, so the experiment runner handles every model in the same way.
(5) *Evaluation.* On the test part of each fold, the scores are compared with the gold labels and the metrics of Section 5.4 are computed. The results of each fold are written to disk as soon as the fold finishes, and we report the mean and standard deviation over the five folds.

### 4.2 Text Representations with a Fixed Classifier

To address RQ1, we represent each comment $x$ as a fixed-length feature vector $\mathbf{z} = \phi(x)$ and apply the same one-vs-rest logistic regression formulation. The representation function $\phi$ is instantiated either as a sparse TF-IDF vectorizer or as a pretrained embedding lookup followed by a pooling operator. This yields one TF-IDF configuration and thirty embedding–pooling combinations. Comparisons across embedding sources are made under a common pooling strategy, while comparisons across pooling strategies are made within each embedding source.

Figure 2 summarizes the representation comparison. Each branch is evaluated separately using the same one-vs-rest logistic regression formulation.

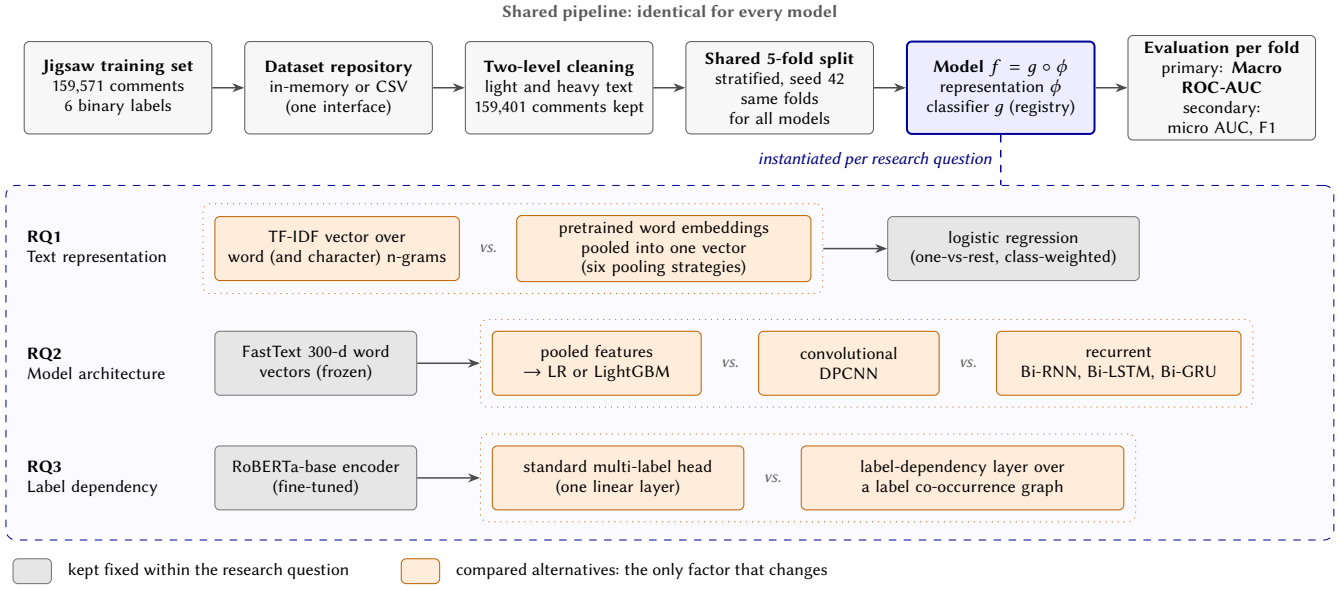



**Figure 1: Overview of the experimental framework. Top: the pipeline that is shared by every model. Bottom: how the model stage is instantiated for each research question. Grey components are kept fixed within a research question; orange components are the alternatives that are compared.**

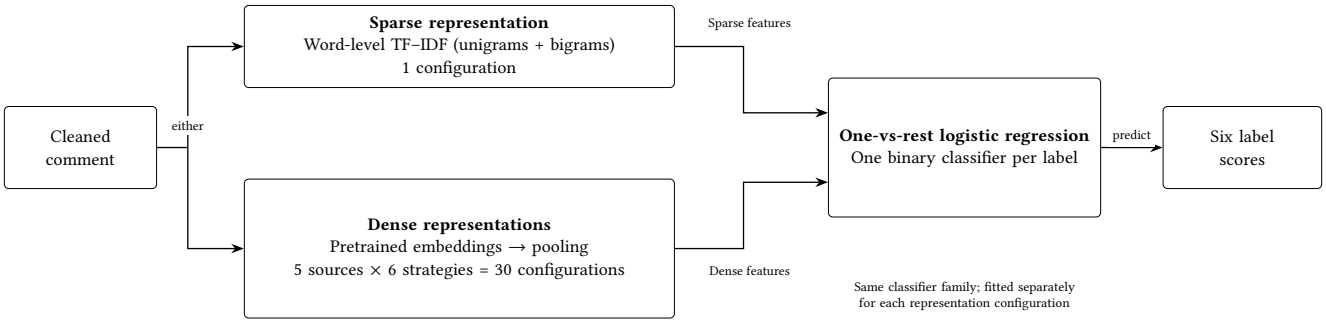



**Figure 2: Representation comparison for RQ1. Each comment is represented using either word-level TF–IDF or a pretrained embedding source combined with a pooling strategy. Each configuration is evaluated separately with one-vs-rest logistic regression, producing six label-specific scores for holdout Macro ROC-AUC evaluation. The two branches denote alternative representations and are not concatenated.**

*4.2.1 TF-IDF Representations.* The TF-IDF baseline represents each comment as a sparse vector of word unigrams and bigrams, with sublinear term-frequency scaling ($1 + \log \mathrm{tf}$). These features capture individual lexical cues and short local expressions without requiring pretrained embeddings. The vocabulary and inverse document frequencies are estimated exclusively from the training partition, and the fitted vectorizer is then applied unchanged to the evaluation partition. Each document vector is L2-normalized.

The resulting features are passed to the same one-vs-rest logistic regression formulation used for the pooled embedding representations. This provides a lexical baseline for assessing whether pretrained embeddings and pooling improve classification performance. Vectorizer settings and classifier hyperparameters are reported in Section 5.3.

*4.2.2 Pretrained Embeddings.*
We consider five pretrained embedding sources: FastText, GloVe Twitter, Word2Vec, LexVec, and BPEmb. These resources provide dense representations derived from external corpora. BPEmb represents text using subword units, whereas the other sources are accessed through their available token vectors.

For a comment containing $T$ retained tokens or subword units, the embedding sequence is

$$\mathbf{E}(x) = (\mathbf{e}_1, \ldots, \mathbf{e}_T), \qquad \mathbf{e}_t \in \mathbb{R}^d,$$

where $d$ depends on the embedding source. The pretrained vectors remain fixed in the RQ1 pipelines. A pooling operator converts

the sequence into a document-level vector suitable for logistic regression.

For word-vector lookup, tokens without available vectors are omitted. A comment with no usable vectors is represented by a zero vector. The embedding resources, dimensionalities, tokenization settings, and sequence-length limits are specified in Section 5.3. Because the sources differ in pretraining corpus, vocabulary, and dimensionality, the comparison concerns the selected pretrained resources rather than isolating the effect of their embedding algorithms.

#### 4.2.3 *Pooling Strategies.*

We investigate six strategies for aggregating token embeddings into a fixed-length document representation.

**Mean pooling.** The document vector is the arithmetic mean of the retained token embeddings:

$$\mathbf{z}_{\text{mean}} = \frac{1}{T}\sum_{t=1}^{T}\mathbf{e}_t.$$

Each retained token contributes equally, providing a simple reference representation.

**Max pooling.** The document vector contains the largest value observed in each embedding dimension:

$$z_{\max,k} = \max_{1\le t\le T} e_{t,k}, \qquad k = 1,\dots,d.$$

This operator preserves prominent coordinate-wise activations rather than averaging them across the comment.

**Corpus-based attention pooling.** A global query vector $\mathbf{q}$ is obtained by L2-normalizing the mean of the available token-vector occurrences in the training corpus. Token weights are computed from their dot-product similarity to this query:

$$\alpha_t = \frac{\exp(\mathbf{e}_t^\top \mathbf{q})}{\sum_{s=1}^{T}\exp(\mathbf{e}_s^\top \mathbf{q})},$$

and the document representation is

$$\mathbf{z}_{\text{att}} = \sum_{t=1}^{T}\alpha_t \mathbf{e}_t.$$

This operator depends on the training corpus but does not use toxicity labels. Its weights reflect alignment with the corpus-derived query, rather than directly learned evidence of toxicity.

**Max–attention pooling.** We concatenate max pooling and corpus-based attention pooling:

$$\mathbf{z}_{\text{max+att}} = \left[\mathbf{z}_{\max}; \mathbf{z}_{\text{att}}\right].$$

This representation combines coordinate-wise maxima with a weighted summary of the token embeddings.

**Supervised attention pooling.** An additive attention mechanism learns token weights from the multi-label training targets. For each token embedding, an attention score is computed as

$$s_t = \mathbf{v}^\top \tanh(\mathbf{W}\mathbf{e}_t + \mathbf{b}),$$

where $\mathbf{W}$, $\mathbf{b}$, and $\mathbf{v}$ are trainable parameters. The normalized weights and pooled vector are

$$\beta_t = \frac{\exp(s_t)}{\sum_{r=1}^{T}\exp(s_r)}, \qquad \mathbf{z}_{\text{sup}} = \sum_{t=1}^{T}\beta_t \mathbf{e}_t.$$

The attention parameters are trained with an auxiliary multi-label classification head using binary cross-entropy, while the pretrained embeddings remain frozen. After this stage, the auxiliary head is discarded, and the learned pooling function generates features for logistic regression. This configuration therefore introduces supervised representation learning without replacing the downstream classifier.

**Max–supervised-attention pooling.** We concatenate max pooling with supervised attention pooling:

$$\mathbf{z}_{\text{max+sup}} = \left[\mathbf{z}_{\max}; \mathbf{z}_{\text{sup}}\right].$$

For this variant, the auxiliary classifier is trained on the concatenated representation, allowing the attention component to be optimized in the presence of max-pooled features.

The individual pooling operators produce $d$-dimensional vectors, whereas the concatenated variants produce (2d)-dimensional vectors. All six operators aggregate their retained embeddings without explicitly modeling token order. Differences in supervision, output dimensionality, and sequence-length handling are considered when interpreting their results.

#### 4.2.4 *Logistic Regression Classifier.*

Each representation is evaluated using six binary logistic regression classifiers in a one-vs-rest formulation. For label $j$, the predicted score is

$$\hat{p}_j(x) = \sigma\left(\mathbf{w}_j^\top \mathbf{z} + b_j\right), \qquad j = 1,\dots,6,$$

where

$$\sigma(a) = \frac{1}{1+\exp(-a)}$$

is the sigmoid function, and $\mathbf{w}_j$ and $b_j$ are the parameters associated with label $j$.

The classifiers use L2 regularization and balanced class weights computed from their respective training targets. Dense pooled embeddings are standardized using training-set feature means and standard deviations before classification. Sparse TF-IDF features retain their vectorizer normalization and are not passed through the dense standardization step. The logistic regression settings and preprocessing choices are documented in Section 5.3.

The six prediction scores are evaluated directly using Macro ROC-AUC; threshold optimization is not required for this comparison. Keeping the classifier formulation fixed provides a common basis for assessing representations, although it does not eliminate differences introduced by preprocessing, feature dimensionality, or supervised pooling.

## 4.3 Model Architectures with Fixed Embeddings

Figure 3 summarizes the three modeling pipelines considered in RQ2. The pooled-feature classifiers operate on aggregated token embeddings, whereas the recurrent and convolutional models encode embedding sequences.

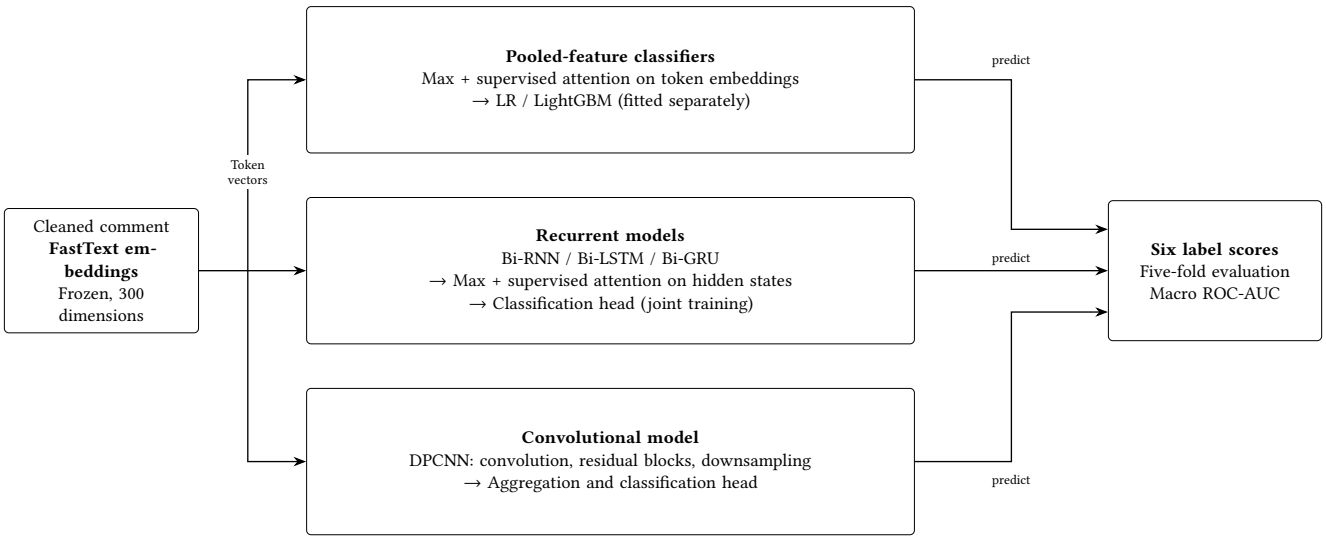



**Figure 3: Modeling pipelines for RQ2 using pretrained FastText embeddings. The pooled-feature branch concatenates max pooling and supervised attention over token embeddings, then fits logistic regression (LR) or LightGBM on the resulting features. The recurrent branch applies max pooling and supervised attention to encoder hidden states, with attention and the classification head trained jointly with the encoder. DPCNN uses a convolutional hierarchy. Each pipeline produces six label-specific scores and is evaluated separately; the branches do not form an ensemble. Supervised pooling is fitted using training data only within each fold.**

#### 4.3.1 *Pooled-Feature Classifiers.*

The pooled-feature branch reuses the representation selected in RQ1. Each comment is mapped to frozen 300-d FastText vectors, and max pooling is concatenated with supervised attention pooling into a 600-d vector. The attention weights are fitted on the training part of each fold only. Two classifiers are trained on these vectors. The first is the one-vs-rest logistic regression of RQ1. The second is LightGBM [12]: one gradient-boosted tree ensemble per label, with 300 trees, learning rate 0.05, 31 leaves, and no class weights. LightGBM can model non-linear interactions between pooled dimensions, but, like logistic regression, it sees only the pooled vector and not the token sequence.

#### 4.3.2 *Deep Pyramid CNN (DPCNN).*

DPCNN [11] is a deep convolutional network for text categorization. Unlike the recurrent models, it processes all positions in parallel and builds long-range context by repeatedly downsampling the sequence. It reads exactly the same input as the recurrent models: the heavy-cleaned comment is padded or truncated to 200 tokens, each token is mapped to its frozen 300-d FastText vector [15], and dropout (0.3) is applied to the embeddings. Only the encoder differs.

**Region embedding and pyramid blocks.** A convolution with kernel size 3 and 250 filters turns every position into a representation of a 3-word region. It is followed by a pre-activation residual unit [9] with two convolutions,

$$\mathbf{H}' = \mathbf{H} + \mathrm{Conv}\big(\mathrm{ReLU}(\mathrm{Conv}(\mathrm{ReLU}(\mathbf{H})))\big). \tag{1}$$

The network then repeats a block of max pooling with kernel size 3 and stride 2, which halves the sequence length, followed by the same residual unit. The number of filters stays fixed at 250, so the cost per block halves at each level. This gives the "pyramid" shape and keeps the total cost below twice that of the first block, however deep the network is. With 200 tokens, 8 blocks reduce the sequence to a single position whose receptive field covers the whole comment.

**Classification and training.** Global max pooling gives a 250-d comment vector. After dropout (0.3), a linear layer with a sigmoid outputs one probability per label. The network has 19 convolutional layers and about 3.6M trainable parameters, not counting the frozen embeddings. Training is identical to the recurrent models: binary cross-entropy, Adam with learning rate $10^{-3}$, batch size 256, 4 epochs, gradient clipping at 1.0, and random seed 42. A label is predicted as positive when its probability is at least 0.5.

#### 4.3.3 *Bi-RNN with Max and Attention Pooling.*

Bag-of-words models ignore word order, so they cannot tell *"you are stupid"* from *"you are not stupid"*. This method reads each comment as a sequence with a vanilla recurrent neural network (RNN), run in both directions [18] (Figure 4). It is the simplest sequence model in this work and has no gates, so it shows how much a plain recurrent encoder gains and how much is left for gated cells (the Bi-LSTM and Bi-GRU variants described at the end of this section).

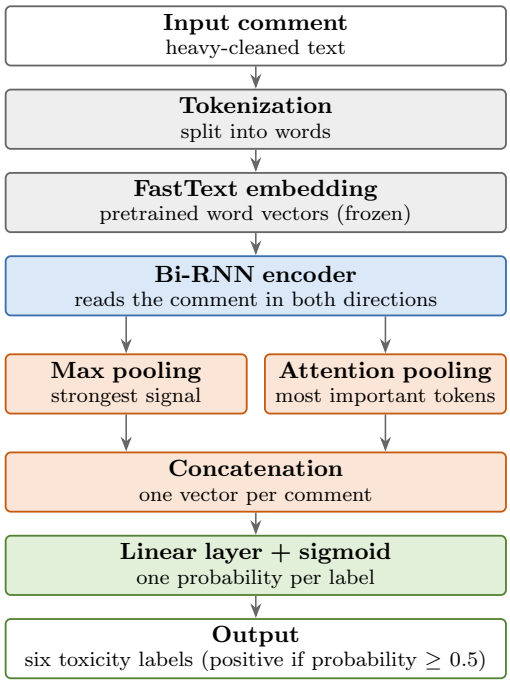



**Figure 4: Bi-RNN with max and attention pooling.**

**Embedding.** The heavy-cleaned comment is split into words and truncated to 200 tokens. The vocabulary (the 100k most frequent words) is built from the training part of each fold only, so no information leaks from the test part. Each word is mapped to a 300-d pretrained FastText vector [15], kept frozen during training. Words not found in FastText receive a random vector drawn from the mean and standard deviation of the found vectors. Dropout (0.3) is applied to the embeddings $\mathbf{x}_1, \ldots, \mathbf{x}_T$.

**Recurrent encoder.** The vanilla RNN updates its hidden state with a single tanh layer and no gates:

$$\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b}). \tag{2}$$

One RNN reads the comment left to right and another right to left, each with 128 units, and their states are concatenated into $\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t] \in \mathbb{R}^{256}$. Sequences are packed, so padding tokens are never processed. Because the gradient is multiplied by $\mathbf{U}$ at every step, it tends to vanish over long comments [1], and the RNN mostly remembers recent words. Reading in both directions partly compensates, since every token is close to one end of either pass.

**Pooling.** The token states are summarized in two ways. Max pooling keeps the strongest signal in each dimension, which suits short toxic phrases. Additive attention pooling [22] learns which tokens matter most:

$$\alpha_t = \mathrm{softmax}_t\big(\mathbf{v}^\top \tanh(\mathbf{W}_a\mathbf{h}_t + \mathbf{b}_a)\big), \qquad \mathbf{a} = \sum_t \alpha_t\, \mathbf{h}_t. \tag{3}$$

Padding positions are masked out of both poolings. The two vectors are concatenated into a 512-d comment representation. Both poolings look at every token state, not only the last one, which also shortens the gradient path to early tokens.

**Classification and training.** After dropout (0.3), a linear layer with a sigmoid outputs one probability per label. The model is trained with binary cross-entropy averaged over the six labels, using Adam (learning rate $10^{-3}$), batch size 256, 4 epochs, gradient clipping at 1.0, and random seed 42. A label is predicted as positive when its probability is at least 0.5.

**Gated variants.** For RQ2 we also train two gated versions of this model, Bi-LSTM and Bi-GRU [3]. They keep exactly the architecture and training setup described above and only replace the tanh cell of Eq. (2) with an LSTM or a GRU cell. The three recurrent models in Table 7 therefore differ only in the recurrent cell. The Bi-GRU variant follows the Bi-GRU with attention architecture, which was among the strongest single models on the Jigsaw data in prior work [19].

## 4.4 Explicit Label-Dependency Modeling

Figure 5 contrasts the baseline classification head with the explicit label-dependency head. The latter exchanges information between label-specific vectors using a graph estimated from training-label co-occurrence.

### 4.4.1 *Standard RoBERTa Classifier.*

To answer RQ3, we train a standard RoBERTa multi-label classifier that shares every component with the label-dependency model (Section 4.4.2) except the classification head (Figure 6). The input comment is cleaned and tokenized in the same way, encoded by the same pretrained `roberta-base` model, and represented by the final hidden state of the first token, $\mathbf{h} = \mathbf{H}_{\langle s \rangle} \in \mathbb{R}^{768}$. Instead of the label-dependency head, a single linear layer maps this vector directly to one logit per label:

$$\hat{\mathbf{y}} = \sigma\left(\mathbf{W}\mathbf{h} + \mathbf{b}\right), \qquad \mathbf{W} \in \mathbb{R}^{6\times 768},\ \mathbf{b} \in \mathbb{R}^6, \tag{4}$$

where $\sigma$ is the element-wise sigmoid. The model is trained end to end with the binary cross-entropy loss

$$\mathcal{L} = -\frac{1}{6}\sum_{l=1}^{6}\left[y_l \log \hat{y}_l + (1-y_l)\log\left(1-\hat{y}_l\right)\right], \tag{5}$$

and label $l$ is predicted positive when $\hat{y}_l \geq 0.5$.

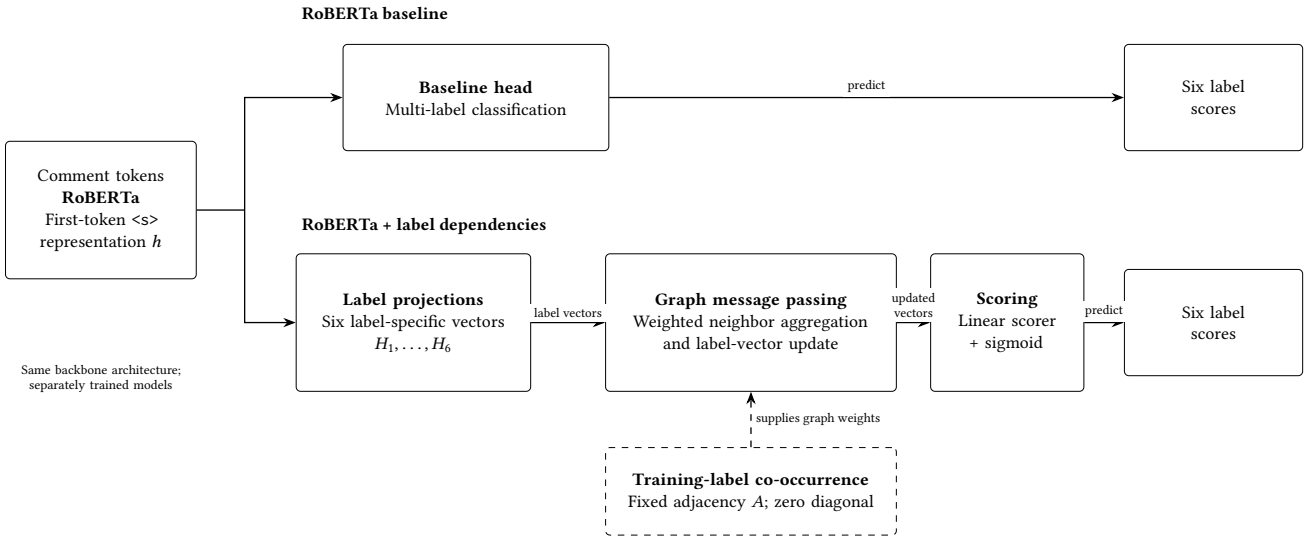



**Figure 5: Comparison of classification heads for RQ3. Both variants use the first-token representation from a RoBERTa encoder. The baseline maps this representation directly to six label scores. The label-dependency variant projects it into six label-specific vectors, performs one graph message-passing step using a fixed adjacency matrix estimated from training-label co-occurrence, and scores the updated vectors. The adjacency has a zero diagonal, while a separate self-transformation retains each label's own information. The variants are trained separately using binary cross-entropy and evaluated using five-fold Macro ROC-AUC; the shared encoder illustration denotes the same backbone architecture, not shared fitted weights. Held-out labels are not used to construct the graph.**

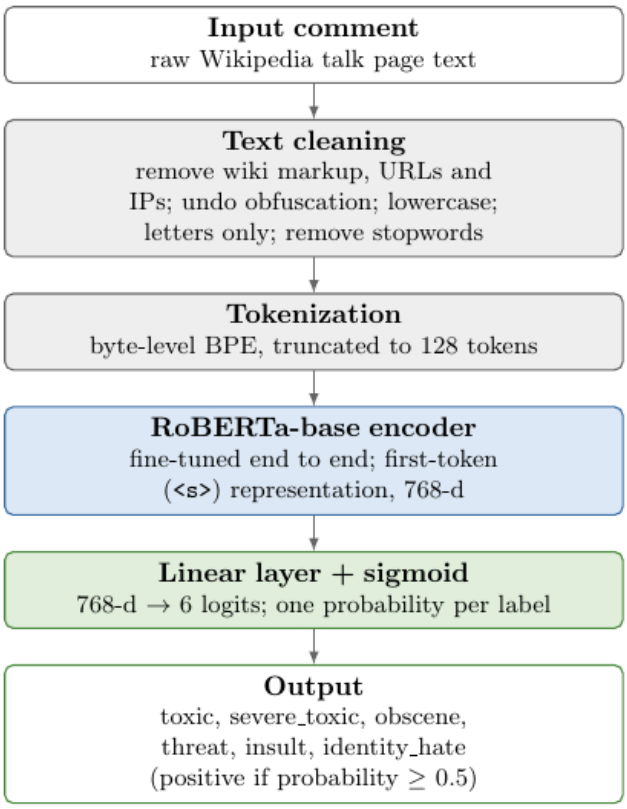



**Figure 6: Standard RoBERTa baseline. It is identical to Figure 7 except that the label-dependency head is replaced by a single linear layer on the <s> representation.**

Both RoBERTa models use identical hyperparameters: a maximum length of 128 tokens, a batch size of 16, AdamW with a learning rate of $2 \times 10^{-5}$, two training epochs, and random seed 42. They are trained and evaluated on the same five stratified folds. Note that the baseline removes the label-dependency head as a whole, that is, the label projection, the self and neighbour transforms, and the shared scorer. The comparison therefore measures the contribution of the complete label-dependency module rather than of the co-occurrence graph alone.

#### 4.4.2 *RoBERTa with a Label-Dependency Layer.*

Our main method extends a RoBERTa multi-label classifier with a small graph layer that models dependencies between the six labels (Figure 7). The encoder, loss, and training setup are the same as in a plain RoBERTa fine-tune, so only the dependency layer is new.

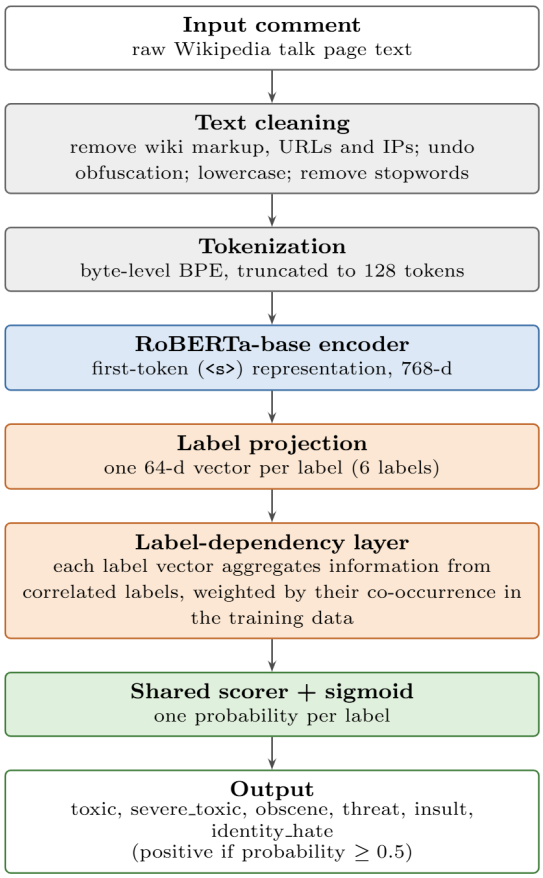



**Figure 7: RoBERTa with the label-dependency layer.**

**Encoder and label projection.** A comment $x$ is encoded by `roberta-base` [14] (max 128 tokens), and the final hidden state of the first token is used as its representation $\mathbf{h} \in \mathbb{R}^{768}$. A linear layer maps $\mathbf{h}$ to six label vectors $\mathbf{z}_l \in \mathbb{R}^{64}$, one per label.

**Label co-occurrence graph.** Edge weights are estimated from the training labels of each fold as

$$A_{ij} = P(y_j = 1 \mid y_i = 1), \qquad A_{ii} = 0, \tag{6}$$

and are kept fixed during training.

**Label-dependency layer.** Following GraphSAGE [8], each label vector is updated with one message-passing step:

$$\mathbf{z}'_l = \mathrm{ReLU}\Big(\mathbf{W}_s \mathbf{z}_l + \mathbf{W}_n \sum_j A_{lj}\, \mathbf{z}_j\Big). \tag{7}$$

This lets a rare label such as *severe_toxic* use evidence from correlated labels such as *toxic*.

**Classification and loss.** A linear scorer shared across all labels maps each updated vector to a logit, and a sigmoid gives the label probability:

$$s_l = \sigma(\mathbf{w}^\top \mathbf{z}'_l + b). \tag{8}$$

The model is trained end-to-end with binary cross-entropy averaged over labels and samples:

$$\mathcal{L} = -\frac{1}{NL} \sum_{n=1}^{N} \sum_{l=1}^{L} \big[ y_{nl} \log s_{nl} + (1 - y_{nl}) \log(1 - s_{nl}) \big]. \tag{9}$$

No class weighting or imbalance-specific loss is used. At inference, a label is predicted as positive when $s_l \geq 0.5$.

**Training details.** All parameters, including the RoBERTa encoder, are fine-tuned jointly with AdamW at a learning rate of $2 \times 10^{-5}$. We train for 2 epochs with a batch size of 16 and a fixed random seed of 42.

## 5 Experimental Setup

### 5.1 Dataset and Preprocessing

**Dataset.** We use the training set of the *Jigsaw Toxic Comment Classification Challenge*: 159,571 English comments from Wikipedia talk pages. Human raters annotated each comment with six non-exclusive binary labels: *toxic*, *severe_toxic*, *obscene*, *threat*, *insult*, and *identity_hate*. An audit of the raw file finds no missing or empty texts, no duplicated identifiers, and no duplicated comments. We do not use the official test set (153,164 comments), because only 63,978 of its comments have labels that were used for scoring. All comparisons instead use cross-validation on the training set.

**Table 1: Label distribution before and after cleaning, ordered by frequency. Percentages are relative to all comments in each version.**

| | Raw (159,571) | | Cleaned (159,401) | |
|---|---|---|---|---|
| Label | # Pos. | % | # Pos. | % |
| toxic | 15,294 | 9.58 | 15,289 | 9.59 |
| obscene | 8,449 | 5.29 | 8,445 | 5.30 |
| insult | 7,877 | 4.94 | 7,873 | 4.94 |
| severe_toxic | 1,595 | 1.00 | 1,594 | 1.00 |
| identity_hate | 1,405 | 0.88 | 1,405 | 0.88 |
| threat | 478 | 0.30 | 478 | 0.30 |
| at least one label | 16,225 | 10.17 | 16,220 | 10.18 |
| no label (clean) | 143,346 | 89.83 | 143,181 | 89.82 |

**Class imbalance.** The data is highly imbalanced (Table 1). Almost 90% of the comments carry no label, and the positive rate ranges from 9.6% for *toxic* to 0.30% for *threat*. Only 478 comments are threats, about 96 per test fold, and for each of them there are about 330 comments without this label. Macro ROC-AUC gives these rare labels the same weight as *toxic*, so they have a large effect on the primary metric.

**Label co-occurrence.** Of the 16,220 comments with at least one label, 9,860 (60.8%) carry two or more. Table 2 shows that *toxic* acts almost as a parent label: every *severe_toxic* comment is also *toxic*, and 93–94% of the comments in each of the other four categories are *toxic* as well. Severe toxicity nearly always comes with obscenity (0.95) and insults (0.86), and *obscene* and *insult* overlap strongly (Pearson correlation 0.74). *Threat* is the least connected label: only 3% of the *toxic* comments are threats. This structure motivates the label-dependency layer studied in RQ3.

**Table 2: Label co-occurrence: each cell gives $P$(column label = 1 | row label = 1) on the cleaned data. Darker cells mean stronger co-occurrence.**

| Given ↓ | tox | obs | ins | sev | idh | thr |
|---|---|---|---|---|---|---|
| toxic (tox) | – | 0.52 | 0.48 | 0.10 | 0.09 | 0.03 |
| obscene (obs) | 0.94 | – | 0.73 | 0.18 | 0.12 | 0.04 |
| insult (ins) | 0.93 | 0.78 | – | 0.17 | 0.15 | 0.04 |
| severe_toxic (sev) | 1.00 | 0.95 | 0.86 | – | 0.20 | 0.07 |
| identity_hate (idh) | 0.93 | 0.73 | 0.83 | 0.22 | – | 0.07 |
| threat (thr) | 0.94 | 0.63 | 0.64 | 0.23 | 0.21 | – |

**Comment length.** Raw comments have a long tail: the median comment has 36 whitespace tokens, the mean is 67, the 95th percentile is 230, and the longest comment has 1,411. Heavy cleaning (below) removes about 44% of all tokens. After it, the median is 20 tokens, the 95th percentile 128, and the 99th percentile 325. Only 2.3% of the comments are longer than 200 tokens, the input length of the sequence models in RQ2, so truncation affects few comments. Toxic comments are shorter than clean ones (median of 14 versus 21 tokens after heavy cleaning).

**Table 3: The two cleaning levels applied to an illustrative comment (⟷ marks a line break).**

| | |
|---|---|
| Raw | `== Reply == ⟷ :You're SUCH an idiooooot!!!!!! Stop it, n.o.w. [[User:Bob\|Bob]] 12:34, 5 May 2006 (UTC) http://example.com` |
| Light | `Reply You're SUCH an idioot!!! Stop it, n.o.w.` |
| Heavy | `reply you idioot stop` |

**Two-level text cleaning.** Wikipedia talk pages contain markup and signatures that carry no toxicity signal, while toxic comments often disguise words through elongation or obfuscation. We therefore clean every comment at two levels (Table 3).

*Light cleaning* removes noise but keeps casing, punctuation, and contractions, which subword tokenizers can use. It decodes HTML entities (twice, since some are double-encoded), applies NFKC Unicode normalization so that, e.g., full-width letters become ASCII, and removes control characters. It strips HTML tags and wiki markup: links to user pages, files, and categories, as well as templates, are removed, while the display text of article links, external links, and section headings is kept. Emphasis quotes and talk-page indentation markers are dropped. It also removes signature timestamps (*"12:34, 5 May 2006 (UTC)"*), URLs, e-mail addresses, and IP addresses. Finally, it shortens runs of a repeated letter to two (*"idiooooot"* becomes *"idioot"*) and runs of repeated punctuation to three.

*Heavy cleaning* is applied on top of the light text and produces normalized words for word-level models. It lowercases the text and strips accents. It rejoins words that are spelled out with spaces, dots, or hyphens (*"f u c k"* and *"f.u.c.k"* become *"fuck"*). Inside a word, the symbols $ and @ are decoded as letters (*"lo$er"* becomes *"loser"*), other mask symbols are deleted (*"st*pid"* becomes *"stpid"*), and digits are removed rather than decoded (*"id10t"* becomes *"id"*). It then expands 49 common contractions, keeps only alphabetic tokens of at least two letters, and removes stopwords from a list of 116 English function words. The list deliberately excludes second-person pronouns (*you*, *your*), negations (*not*, *never*, *no*), and some intensifiers, because personal attacks are often addressed to "you" and a negation can reverse the meaning of a comment.

All models in this report take the heavy text as input, including RoBERTa, whose byte-level BPE tokenizer is applied to the heavy text. The light text is kept in the processed data for subword models.

**Removed comments.** Exact duplicates (same text and labels) would be dropped, but the data contains none. Ninety-five comments become empty after light cleaning: they contain only a signature timestamp, a URL or IP address, or text that is fully enclosed in a wiki template. Four of them carry a toxicity label. These comments are removed. Another 75 comments become empty after heavy cleaning because they contain only numbers, punctuation, or stopwords (e.g., *"27"* or *"Where was it before?"*); one of them is toxic. Word-level models cannot represent an empty text, so these comments are excluded as well. The final dataset contains 159,401 comments (99.89% of the original), and the label distribution is practically unchanged (Table 1).

**Stored data.** The cleaning step also derives 15 surface features from the raw text, such as the length, the ratio of capital letters, the number of second-person pronouns, and the number of masked words. They are stored with the processed data but are not part of the representations compared in RQ1–RQ3. The processed file keeps the raw, light, and heavy text side by side. The experiment runner recognizes an already cleaned file and does not clean it again, so every model reads identical text.

## 5.2 Data Splitting and Validation Protocol

We do not use the official Jigsaw test set (Section 5.1). All models are instead evaluated with 5-fold cross-validation on the 159,401 cleaned training comments. The folds are created once, before any model is trained, and are stored on disk (`fold_k/train.csv` and `fold_k/test.csv`). Every model therefore reads exactly the same train and test indices.

**Multi-label stratification.** Standard stratified $k$-fold assumes one label per example. We therefore treat each distinct combination of the six labels as one stratum, for example `100000` for comments that are only *toxic*. Combinations with fewer than five comments, too few to place one in every fold, are merged into a single stratum for rare label combinations. The strata are split with scikit-learn's `StratifiedKFold` (5 folds, shuffling, seed 42). Each test fold then holds about 31,900 comments and keeps the label distribution of Table 1. In particular, it contains about 96 *threat* and 319 *severe_toxic* positives, so per-label ROC-AUC can be computed for every label in every fold. Stratifying on label combinations also preserves the co-occurrence structure of Table 2 within each fold.

**No leakage.** Everything learned from data is fitted on the training part of a fold only. This includes TF-IDF vocabularies and IDF weights, word vocabularies for the neural models, feature standardization statistics, and the supervised attention weights of the pooled-feature models, and the label co-occurrence graph of the

**Table 4: Settings of the pooled-feature models (RQ1 and RQ2).**

| Component | Setting |
|---|---|
| TF-IDF | word 1–2-grams, at most 50k features, sublinear tf, lowercased, accents stripped, tokens of at least two characters |
| Embeddings | FastText wiki-news (300-d), GloVe Twitter (200-d), word2vec Google News (300-d), LexVec Common Crawl (300-d), BPEmb English (10k subwords, 300-d) |
| Pooling | mean, max, attention, max + attention, supervised attention, max + supervised attention |
| Logistic regression | one-vs-rest, $C = 4$, liblinear solver, balanced class weights, at most 1000 iterations; embedding features are standardized |
| LightGBM | one model per label, 300 trees, learning rate 0.05, 31 leaves, no class weights |

label-dependency layer. The test part of a fold is used only to compute the metrics in Section 5.4. Hyperparameters are fixed in advance and are not tuned on the test folds, and every model uses the same fixed decision threshold of 0.5.

**Reporting.** For each model we report the mean and standard deviation of every metric over the five test folds. All models share the same folds, so two models can also be compared fold by fold. We use this paired comparison in RQ2 and RQ3 (Section 6).

**Exceptions.** Two experiments deviate from this protocol. First, the RQ1 embedding–pooling grid (Table 6) is evaluated on a single stratified 80/10/10 train/validation/test split of 159,476 comments (seed 42): 127,580 training and 15,948 validation comments, built with the same label-combination strata. The 10% test part is not used. This was done because thirty configurations had to be evaluated. Its scores are therefore not directly comparable with the cross-validated scores in Tables 7–9. Second, the RoBERTa folds were generated before the 75 comments that become empty after heavy cleaning were removed (Section 8). They contain 159,476 comments, a difference of under 0.05%.

## 5.3 Model Configurations and Training Procedure

All models are trained only on training data and evaluated on held-out data, following the protocol of Section 5.2, and all random seeds are fixed to 42. Table 4 lists the settings of the pooled-feature models used for RQ1 and RQ2, and Table 5 those of the neural models used for RQ2 and RQ3.

**Pooled-feature models.** For the unsupervised pooling strategies, the comment vector is computed directly from the frozen word vectors. The supervised attention weights are learned first: a small attention network with frozen embeddings is trained on the training labels for 3 epochs (Adam, learning rate $10^{-3}$, batch size 256, dropout 0.1, at most 200 tokens). The learned weights are then used to pool each comment, and the classifier is trained on the resulting vectors.

**Neural models with fixed embeddings.** The three recurrent models and DPCNN share the input: the vocabulary consists of the 100k most frequent tokens of the training part of each fold, and each token is mapped to its frozen 300-d FastText vector. About 61.8% of the vocabulary is found in FastText; the remaining tokens receive random vectors drawn from a normal distribution with the mean and standard deviation of the found vectors. The recurrent models differ only in the recurrent cell, and DPCNN replaces the recurrent encoder and its pooling with the pyramid of convolutions (Table 5).

**RoBERTa models.** Both RoBERTa models use the roberta-base checkpoint and its byte-level BPE tokenizer, and they are fine-tuned end to end. They share the tokenizer, the first-token representation, the optimizer, and all training settings, and differ only in the output head: the standard model applies one linear layer to the 768-d representation, while the label-dependency model applies the layer described in Section 4.4.2. The label co-occurrence matrix of that layer is estimated from the training labels of each fold.

**Shared training settings.** All neural models are trained with binary cross-entropy averaged over the six labels and without class weights. Logistic regression uses balanced class weights, and LightGBM uses none. For every model, a label is predicted as positive when its probability is at least 0.5. The models were trained on different machines, so their training times are not compared.

## 5.4 Evaluation Metrics

All research questions are evaluated with macro-averaged metrics, so that every label has the same weight and the rare labels (threat, identity_hate, severe_toxic) count as much as the frequent ones.

**Main metric: Macro ROC-AUC.** For each label $l$, ROC-AUC is the probability that a randomly chosen positive comment receives a higher score than a randomly chosen negative one,

$$\mathrm{AUC}_l = P\big(s_l(x^+) > s_l(x^-)\big), \tag{10}$$

$$\text{Macro ROC-AUC} = \frac{1}{L}\sum_{l=1}^{L} \mathrm{AUC}_l, \tag{11}$$

where ties count as one half. Macro ROC-AUC is the official metric of the Jigsaw challenge. It measures how well a model ranks comments and does not depend on the decision threshold, so models trained with and without class weights can be compared fairly.

**Secondary metrics.** To show how the models behave at the fixed threshold of 0.5, we also report Macro-F1, the unweighted mean of the per-label F1 scores, together with macro precision and recall where they help to explain a result. Per-label ROC-AUC and F1 are reported to analyze the rare labels. Micro ROC-AUC and Micro-F1, which pool all (comment, label) pairs and are therefore dominated by the frequent labels, are given for completeness only.

**Aggregation.** All metrics are computed on held-out data that the model has not seen during training (Section 5.2). For RQ1, they are computed once on the hold-out validation split. For RQ2 and RQ3, they are computed on the test part of each of the five folds and reported as the mean and standard deviation over the folds; since the models of each comparison share the same folds, two models can also be compared fold by fold.

## 5.5 Implementation and Reproducibility

*5.5.1 Project organization.*

**Table 5: Configuration of the neural models (RQ2 and RQ3). Parameter counts exclude the frozen word embeddings.**

| | Bi-RNN / Bi-LSTM / Bi-GRU | DPCNN | RoBERTa | RoBERTa + LD |
|---|---|---|---|---|
| Input | FastText 300-d, frozen, 200 tokens | | byte-level BPE, 128 tokens | |
| Encoder | 1 layer, tanh / LSTM / GRU cell | 9 residual blocks | RoBERTa-base, 12 layers | |
| Width | 128 per direction | 250 filters | 768 | |
| Pooling | max + attention | global max | first token | |
| Output head | linear | linear | linear | LD layer ($6 \times 64$-d) |
| Comment vector | 512-d | 250-d | 768-d | 768-d |
| Dropout | 0.3 | 0.3 | 0.1 (encoder) | 0.1 (encoder) |
| Optimizer | Adam, learning rate $10^{-3}$ | | AdamW, learning rate $2 \times 10^{-5}$ | |
| Batch size / epochs | 256 / 4 | | 16 / 2 | |
| Gradient clipping | 1.0 | | none | |
| Trainable parameters | 0.18M / 0.51M / 0.40M | 3.61M | 125M | 125M |

The application is implemented in Python and separates reusable source code, datasets, interactive experiments, trained models, evaluation outputs, and tests. The installable package resides in `src/toxic_comments/`. The `data/` directory contains raw datasets, processed datasets, reusable fold files, and external pretrained embeddings. The `notebooks/` directory provides workflows for exploratory analysis, cleaning, embedding comparisons, and neural-model experiments. Fitted model artifacts are stored in the root-level `models/` directory, whereas model implementations reside in `src/toxic_comments/models/`. Metrics, summaries, logs, and figures are stored in `results/`; automated tests and report materials are maintained in `tests/` and `report/`, respectively. Figure 8 summarizes the execution paths and dataset-access abstraction.

*5.5.2 Application components and execution.*

Within the package, `config.py` centralizes label names, text columns, and directory paths; `cleaning.py` implements text transformations; and `splits.py` generates holdout or k-fold partitions and exports reusable fold CSV files. The `embeddings/` subpackage provides pretrained-vector loaders and pooling operators, while `build_models` in the model registry constructs named estimators. Shared evaluation utilities extract continuous scores through `predict_proba` or `decision_function` and compute aggregate and per-label metrics.

The repository-based `run_experiment` workflow prepares data, selects models, generates shared fold indices, and invokes `cross_validate_model`, which clones and fits an estimator for each fold. A complementary workflow in `train.py` provides `train_model`, `train_holdout`, and `train_from_folds`, together with a full-dataset training mode. Both workflows reuse model construction and evaluation utilities, but differ in orchestration and persistence. The `predict.py` and `submission.py` modules support inference from saved estimators and generation of six-label submission files, respectively.

*5.5.3 Interchangeable dataset backends.*

The `DatasetRepository` protocol defines `load()`, returning a pandas DataFrame, and `save(data)`, storing a DataFrame. `InMemory DatasetRepository` maintains data in memory and copies it on loading and saving, while `CsvFileDatasetRepository` implements the same operations using filesystem CSV files. The repository is supplied to `run_experiment` as an argument. Switching backends therefore changes only the repository instance at the call site; the DataFrame representation and subsequent validation, preprocessing, training, and evaluation logic remain unchanged.

This abstraction addresses the requirement for interchangeable memory and file-based dataset management. Its scope is dataset access in the repository-based workflow: other entry points can read CSV files directly, and model and metric persistence remain filesystem-based. The runner currently consumes the `load()` operation; both backends additionally expose `save(data)` for dataset updates.

*5.5.4 Persistence and experiment traceability.*

The `save_kfold_datasets` utility writes reusable `fold_N/train.csv` and `fold_N/test.csv` partitions. Training utilities serialize fitted estimators with `joblib`, including fold-specific models, and save aggregate and per-label metrics separately. The repository-based cross-validation workflow writes completed fold metrics incrementally, preserving those records if later computation is interrupted. This does not automatically resume unfinished training. Separate output directories are required to retain distinct runs because some workflows replace existing outputs.

Settings are supplied through function arguments, command-line options, and notebook cells. The embedding-comparison notebook additionally computes experiment fingerprints from package source code, supplied settings, and the relevant training and validation data. Result records carry these fingerprints, allowing completed configurations to be recognized during resumed experiment sweeps. Source-change checks require a kernel restart when imported code may be stale. This mechanism is specific to that notebook and does not capture the complete software environment or all external resource versions.

*5.5.5 Testing and reproducibility.*

The pytest suite covers cleaning, split generation, score extraction, metric aggregation, model registration, embedding and neural components, threshold tuning, training, prediction, and submission formatting. Existing tests check reproducible split generation, holdout disjointness, fold-file creation, and model persistence using small datasets and temporary directories. These tests support regression detection; they do not substitute for predictive evaluation or establish that the complete suite passes in every environment.

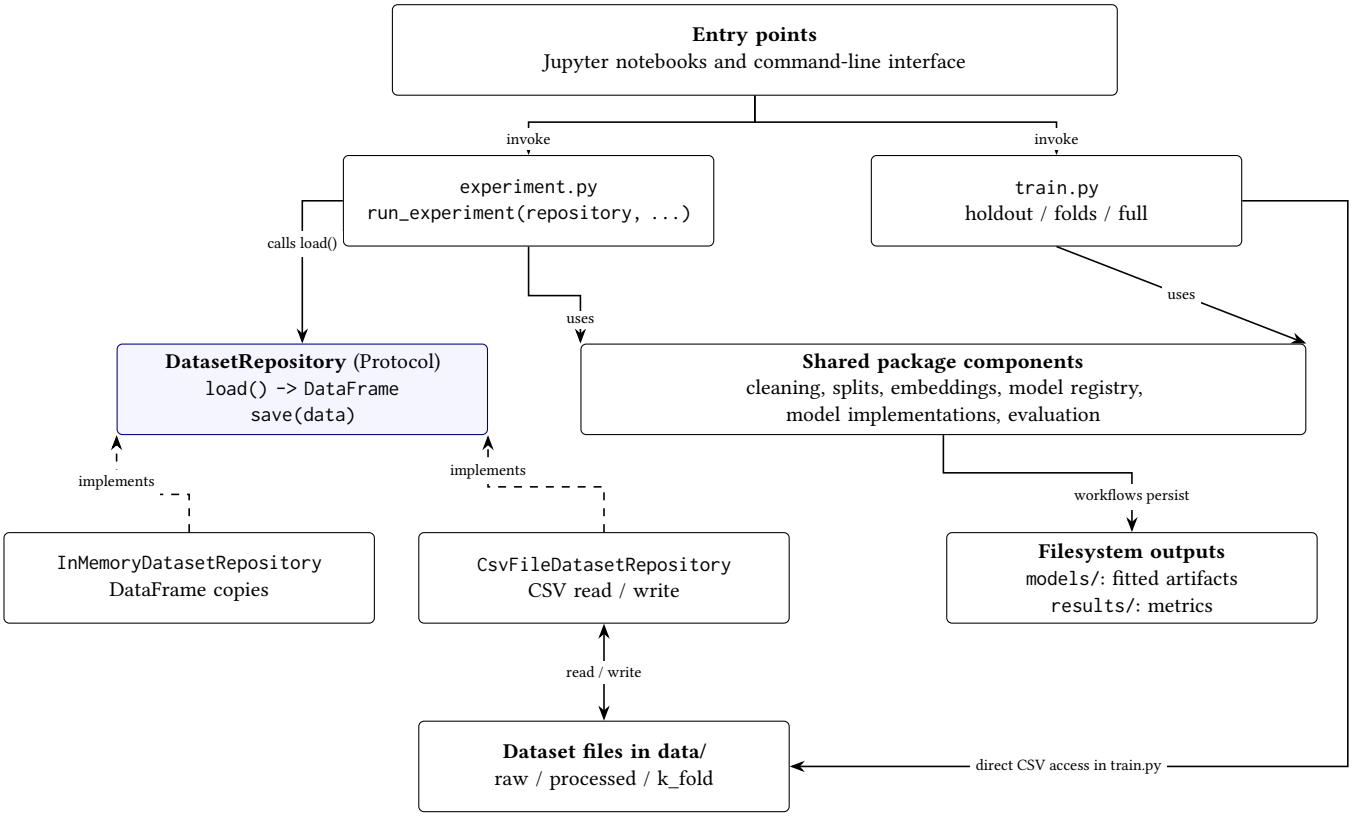



**Figure 8: Software architecture and interchangeable dataset backends. The repository-based experiment runner consumes the same DataFrame contract for memory and CSV storage. Dashed arrows indicate interface implementation; solid arrows indicate calls or access. The training workflow also supports direct CSV access. Models and results remain filesystem-based artifacts.**

Random-state settings support repeatable partitioning and stochastic model components. Dependencies are declared in `pyproject.toml` and `requirements.txt`, including optional groups for additional model families. These declarations use version constraints rather than a fully locked environment. Consequently, exact reproduction additionally requires the resolved dependency versions, hardware details, pretrained resource versions, data partitions, and model settings. Fixed seeds alone do not guarantee identical neural-network outputs across environments. Git tracks source changes and development contributions; reported runs should be associated with a commit and their execution configuration so that saved metrics and model artifacts can be traced to the corresponding implementation.

# 6 Results

## 6.1 RQ1: Effect of Text Representation

RQ1 investigates how TF-IDF representations and pooled pretrained embeddings compare when used with logistic regression. Table 6 reports the word-level TF-IDF baseline alongside five embedding sources combined with six pooling strategies. Pooling is applied only to pretrained embeddings.

The embedding experiments use 127,580 training comments and 15,948 validation comments, with `comment_heavy` as the input text. The TF-IDF baseline is fitted and evaluated in the same notebook run on the same split (Section 5.2). These results are holdout validation scores rather than cross-validation averages.

**Effect of the embedding source**

The choice of pretrained embedding affects classification performance even when the pooling strategy and downstream classifier are fixed. Under mean pooling, FastText achieves the highest Macro ROC-AUC of 0.97245, followed by Word2Vec at 0.96674, GloVe Twitter at 0.96619, and LexVec at 0.96582. BPEmb obtains 0.95366. The difference between the highest- and lowest-scoring sources is therefore 0.01879 AUC units under this common aggregation strategy.

With max–supervised-attention pooling, FastText again achieves the highest reported score of 0.97956, followed by GloVe Twitter at 0.97779. Word2Vec, LexVec, and BPEmb obtain 0.97147, 0.96715, and 0.95612, respectively. Although FastText ranks first under both

**Table 6: Validation Macro ROC-AUC for TF-IDF and pooled pretrained embeddings with logistic regression. Pooling is not applicable to TF-IDF. Bold values indicate the highest reported score within each embedding source.**

| Representation | Mean | Max | Attention | Max + attention | Supervised attention | Max + supervised attention |
|---|---|---|---|---|---|---|
| *Sparse lexical baseline* | | | | | | |
| Word TF-IDF | 0.97541 (pooling not applicable) | | | | | |
| *Pooled pretrained embeddings* | | | | | | |
| FastText | 0.97245 | 0.97214 | 0.97255 | 0.97323 | 0.94219 | **0.97956** |
| GloVe Twitter | 0.96619 | 0.95637 | 0.94475 | 0.96338 | 0.97252 | **0.97779** |
| BPEmb | 0.95366 | 0.92818 | 0.93300 | 0.94242 | 0.88638 | **0.95612** |
| Word2Vec | 0.96674 | 0.95389 | 0.96881 | 0.96870 | 0.95548 | **0.97147** |
| LexVec | 0.96582 | 0.94145 | 0.95801 | 0.95835 | 0.95633 | **0.96715** |

*Notes.* Attention denotes corpus-based, label-independent pooling. Supervised attention is trained using the multi-label targets before fitting the downstream logistic regression classifier. Scores are from a single holdout validation split shared by all configurations, not cross-validation averages.

strategies, GloVe Twitter overtakes Word2Vec when max–supervised-attention pooling is used. The relative performance of embedding sources therefore depends partly on the aggregation strategy.

These observations suggest that embedding selection and pooling should be considered jointly. However, the differences cannot be attributed exclusively to the embedding algorithms, because the selected resources also differ in pretraining corpus, vocabulary, dimensionality, and tokenization. The findings therefore concern the particular pretrained resources evaluated in this study rather than a universal ranking of embedding methods.

**Effect of the pooling strategy**

More complex pooling does not consistently improve performance. Max pooling underperforms mean pooling for all five embedding sources. The decrease is small for FastText, at 0.00031 AUC units, but substantially larger for BPEmb and LexVec, at 0.02548 and 0.02437, respectively. Retaining coordinate-wise maxima alone is therefore less effective than averaging the token embeddings in these configurations.

Corpus-based attention also has mixed effects. Compared with mean pooling, it produces a small increase of 0.00010 for FastText and a larger increase of 0.00207 for Word2Vec, while reducing performance for GloVe Twitter, BPEmb, and LexVec. Combining max pooling with corpus-based attention exceeds mean pooling only for FastText and Word2Vec, with gains of 0.00078 and 0.00196, and remains below mean pooling for GloVe Twitter, BPEmb, and LexVec. These results show that the benefits of corpus-derived weighting and feature concatenation depend on the embedding source.

Standalone supervised attention improves GloVe Twitter from 0.96619 to 0.97252, but falls below mean pooling for the other four sources. The largest decrease occurs for BPEmb, whose score drops from 0.95366 to 0.88638. Consequently, introducing label supervision into the pooling stage is not sufficient by itself to guarantee an improvement. The observed performance may also depend on optimization settings, sequence-length handling, and the interaction between the attention mechanism and the underlying embeddings.

In contrast, max–supervised-attention pooling obtains the highest reported score within every embedding source. Relative to mean pooling, the gains are 0.00711 for FastText, 0.01160 for GloVe Twitter, 0.00246 for BPEmb, 0.00473 for Word2Vec, and 0.00133 for LexVec. This pattern is consistent with the possibility that max-pooled features and supervised attention provide complementary information. Nevertheless, the combined representation also doubles the feature dimensionality and changes the input used to train the auxiliary attention classifier. The observed gains therefore cannot be attributed solely to the attention weights.

**Comparison with the TF-IDF baseline**

The word-level TF-IDF logistic regression baseline achieves a validation Macro ROC-AUC of 0.97541. Only two of the thirty embedding–pooling configurations exceed this score: FastText and GloVe Twitter with max–supervised-attention pooling, at 0.97956 and 0.97779, corresponding to gains of 0.00415 and 0.00238 AUC units.

The strongest configurations for Word2Vec, LexVec, and BPEmb remain below the TF-IDF baseline. Thus, externally pretrained representations do not uniformly outperform sparse lexical features when both are paired with logistic regression. A possible explanation is that TF-IDF retains specific lexical indicators that may be diluted when token embeddings are aggregated into a single document vector. This interpretation is plausible but would require additional feature-level or error analysis to verify.

The comparison with TF-IDF rests on a single split and a single seed, so gains of a few thousandths should be read with caution. Moreover, a matched word–character TF-IDF result is not included in the current table. The available results therefore do not establish whether the highest-scoring pooled embedding also outperforms a character-enriched TF-IDF baseline.

**Configuration selection for subsequent experiments**

Based on these results, we select FastText with max–supervised-attention pooling for subsequent experiments. This combination achieves the highest validation Macro ROC-AUC among the evaluated configurations, at 0.97956, improving over FastText mean pooling by 0.00711 and exceeding the word-level TF-IDF baseline by 0.00415. Furthermore, max–supervised-attention pooling yields the best result for each of the five embedding sources, supporting its selection as the aggregation strategy. FastText is then chosen as the embedding source because it achieves the highest score with this strategy. This selection is based on the observed validation

performance within the evaluated configurations; it does not imply statistically significant superiority or a universal advantage across datasets.

## 6.2 RQ2: Effect of Model Architecture

**Controlled setting.** All models in this comparison start from the same pretrained word vectors, the FastText `wiki-news-300d-1M` vectors [15], kept frozen. All of them read the same heavy text and use the same five folds. The pooled-feature classifiers use the pipeline selected in RQ1: max pooling concatenated with supervised attention pooling into a 600-d vector, with the attention weights fitted on the training part of each fold. This vector is then classified either by one-vs-rest logistic regression with balanced class weights or by LightGBM with 300 trees per label and no class weights. The neural models build a vocabulary of the 100k most frequent heavy-text tokens from the training part of each fold. About 61.8% of these word types are found in FastText, and the others receive fixed random vectors. The neural models share one training setup: binary cross-entropy without class weights, Adam with learning rate $10^{-3}$, batch size 256, 4 epochs, gradient clipping at 1.0, inputs truncated to 200 tokens, and random seed 42. For every model, threshold-dependent metrics use a fixed threshold of 0.5. The models therefore differ mainly in how they combine the token vectors of a comment into one score per label.

**Overall ranking.** Table 7 reports the results. The clearest effect separates the two pooled-feature classifiers from the four sequence models. Logistic regression on the pooled vectors reaches a macro ROC-AUC of 0.9715 and LightGBM 0.9800, whereas the sequence models reach 0.9851 to 0.9877. Paired over the five folds, DPCNN and Bi-GRU each beat LightGBM by 0.0077 and logistic regression by about 0.016, on every fold. LightGBM in turn beats logistic regression by $0.0085 \pm 0.0019$, also on every fold, so a non-linear classifier recovers about half of the gap between a linear model on pooled vectors and the best sequence models. Among the sequence models, macro ROC-AUC lies in a narrow band, from 0.9851 for Bi-RNN to 0.9877 for both DPCNN and Bi-GRU. DPCNN and Bi-GRU are tied: paired over the five folds, their difference is $0.0000 \pm 0.0007$, and DPCNN is better on three of the five folds.

**Learned composition matters more than the encoder type.** A pooled vector keeps the strongest value in each dimension and a learned weighted average of the token vectors, but it ignores word order and cannot combine neighbouring words. The sequence models learn this composition, either with recurrence or with convolution. Even the weakest of them, Bi-RNN (0.9851), stays 0.0051 above LightGBM, about twice the spread among the four sequence models (0.0026).

**Gated recurrence helps consistently.** The three recurrent models are identical except for the recurrent cell, and their ranking Bi-RNN < Bi-LSTM < Bi-GRU holds on every fold. Paired over the folds, Bi-LSTM beats Bi-RNN by $0.0015 \pm 0.0006$ and Bi-GRU beats Bi-LSTM by $0.0010 \pm 0.0003$. The gain of Bi-GRU over Bi-RNN ($0.0025 \pm 0.0007$) is two to four times the fold standard deviation of either model and is positive on all five folds. This agrees with the motivation for gated cells: gates let the gradient pass through long sequences without vanishing [1, 3]. The vanilla RNN also fits the training data worst: after four epochs its training loss is 0.046 on every fold, against 0.044 for Bi-LSTM and 0.043 for Bi-GRU. Bi-LSTM has the most parameters of the three (0.51M) but stays behind Bi-GRU. It starts more slowly (training loss 0.086–0.088 after the first epoch, against 0.076–0.077 for Bi-GRU), so it may not have converged within the fixed budget of four epochs.

**Convolution versus recurrence.** DPCNN reaches the same macro ROC-AUC as Bi-GRU with a very different design. It combines local windows with stacked convolutions and halves the sequence eight times, so that its last layer covers the whole 200-token input, whereas the Bi-GRU reads the comment in both directions and pools all token states with max and attention pooling. DPCNN has about nine times more trainable parameters (3.61M versus 0.40M) but gains nothing in ranking quality. Among learned encoders that can keep information across long inputs, the type of encoder therefore makes no measurable difference; what matters is avoiding the weak vanilla RNN.

**Where architecture matters: the rarest labels.** The differences are not spread evenly over the labels. Between Bi-GRU and the pooled logistic regression, the three rarest labels (*severe_toxic*, *identity_hate*, *threat*) account for 78% of the gap in macro ROC-AUC, and *threat* alone for 42%: its ROC-AUC rises from 0.9450 (LR) and 0.9699 (LightGBM) to 0.9861 (Bi-GRU). The same pattern holds among the sequence models. On the three frequent labels (*toxic*, *obscene*, *insult*), their per-label ROC-AUC differs by at most 0.002, while on *threat*, which has only about 96 positives per test fold, it ranges from 0.9772 (Bi-RNN) over 0.9816 (Bi-LSTM) and 0.9861 (Bi-GRU) to 0.9867 (DPCNN). This single label accounts for 58% of the gain of Bi-GRU over Bi-RNN and for 62% of the gain of DPCNN over Bi-RNN. Because macro ROC-AUC weights all labels equally, the architecture effect in RQ2 is mainly an effect on how well a model ranks the few comments of the rare labels. These are also the least stable estimates: for the sequence models, the standard deviation of the ROC-AUC on *threat* is 0.003–0.006, against at most 0.001 on *toxic*.

**Ranking versus decisions.** The threshold-dependent metrics in Table 7 also depend on how each model's scores are calibrated. The class-weighted logistic regression pushes many comments above the 0.5 threshold: it has the highest micro recall of all models (0.919) but the lowest micro precision (0.416), and therefore the lowest macro-F1 (0.4428). The other models use no class weights. Among the sequence models, macro-F1 ranges from 0.5182 to 0.6259, a spread of 0.108, about 40 times their spread in macro ROC-AUC (0.0026). The F1 on *threat* is 0.040 for Bi-RNN, 0.077 for Bi-LSTM, 0.285 for Bi-GRU, and 0.404 for DPCNN, although all four rank threats well (ROC-AUC of at least 0.977). At the 0.5 threshold, Bi-RNN labels on average only 4 comments per test fold as threats, while each fold contains about 96; Bi-GRU labels 32 and DPCNN 61. Without class weights, rare positives receive low probabilities, and a fixed threshold hides them. DPCNN tends to give higher scores to positives, which yields a higher micro recall than the recurrent models (0.793 against 0.672–0.707) and a lower micro precision (0.749 against 0.822–0.836). LightGBM even matches Bi-LSTM on macro-F1 (0.5381) and exceeds Bi-RNN, although its macro ROC-AUC is clearly lower. The F1 differences thus reflect score calibration more than ranking quality. This supports macro ROC-AUC as the primary metric, and per-label thresholds would likely narrow the F1 gaps.

**Table 7: RQ2: effect of model architecture with a fixed embedding source (FastText `wiki-news-300d-1M`, frozen). Mean ± standard deviation over the same five folds. Macro ROC-AUC is the primary metric. Trainable parameters are counted from the fitted models and exclude the frozen embedding matrix. Best value per column in bold.**

| Family | Model | Params | **Macro ROC-AUC** | Micro ROC-AUC | Micro-F1 | Macro-F1 |
|---|---|---|---|---|---|---|
| Pooled features | LR | 0.09M | 0.9715 ± 0.0009 | 0.9801 ± 0.0002 | 0.5722 ± 0.0026 | 0.4428 ± 0.0027 |
| Pooled features | LightGBM | N/A[†] | 0.9800 ± 0.0023 | 0.9830 ± 0.0008 | 0.7347 ± 0.0042 | 0.5381 ± 0.0051 |
| Convolutional | DPCNN | 3.61M | **0.9877** ± 0.0009 | 0.9906 ± 0.0003 | **0.7694** ± 0.0048 | **0.6259** ± 0.0120 |
| Recurrent | Bi-RNN | 0.18M | 0.9851 ± 0.0011 | 0.9896 ± 0.0006 | 0.7422 ± 0.0227 | 0.5182 ± 0.0382 |
| Recurrent | Bi-LSTM | 0.51M | 0.9866 ± 0.0006 | 0.9903 ± 0.0004 | 0.7579 ± 0.0069 | 0.5381 ± 0.0244 |
| Recurrent | Bi-GRU | 0.40M | **0.9877** ± 0.0007 | **0.9907** ± 0.0004 | 0.7623 ± 0.0062 | 0.5941 ± 0.0207 |

LR: 3.6k classifier weights plus 90.6k supervised-attention parameters. [†]LightGBM: the same 90.6k attention parameters plus 1,800 trees (300 per label), not comparable with a neural parameter count.

**Answer to RQ2.** With the embedding source fixed, the largest effect comes from replacing a fixed pooling of word vectors with a learned sequence encoder: macro ROC-AUC rises from 0.9715 (logistic regression) and 0.9800 (LightGBM) to 0.9851–0.9877, and the best sequence models beat both pooled-feature classifiers on every fold. Among the learned sequence encoders, the choice changes macro ROC-AUC by at most 0.0026. Gated recurrent cells beat a vanilla RNN on every fold, and a deep convolutional network with nine times more parameters matches the best recurrent network. Most of these differences come from the rarest labels, above all *threat*.

### 6.3 RQ3: Effect of Label-Dependency Modeling

We compare two heads on the same fine-tuned `roberta-base` encoder (Table 8). The standard linear head scores each label independently from the first-token (<s>) representation. The label-dependency head performs one message-passing step over a label graph built from training-fold co-occurrences. All other settings are identical.

On ranking, the two heads are practically equivalent: the label-dependency head reaches a macro ROC-AUC of 0.9881 ± 0.0011 versus 0.9887 ± 0.0014 for the linear head. This difference of −0.0006 lies well within fold-to-fold variation. Micro-F1 is also unchanged (0.7745 vs. 0.7742). The only gain appears in macro-F1 (+0.0105, 0.6189 vs. 0.6084), but Table 9 shows that it comes almost entirely from *threat* (+0.0988), the rarest label, on which the linear head is highly unstable across folds. On *severe_toxic* and *identity_hate*, the label-dependency head is worse. We attribute the small effect to the fine-tuned encoder: since all labels share a jointly optimized <s> representation, label correlations are already captured implicitly.

**Answer to RQ3.** Explicit label-dependency modeling does not improve ranking quality over a linear head on a fine-tuned RoBERTa encoder. It yields only a small macro-F1 gain that comes from a single rare label, at a cost of about 0.3M extra parameters (≈0.24% of the model).

## 7 Analysis and Discussion

### 7.1 RQ1: Embedding Quality Depends on the Pooling Strategy

The representation comparison shows that the effectiveness of pretrained embeddings depends strongly on how token vectors are aggregated. FastText with max–supervised-attention pooling achieves the highest Macro ROC-AUC in this comparison, at 0.97956. GloVe Twitter with the same pooling strategy follows at 0.97779. These scores exceed the TF-IDF baseline of 0.97541 by 0.00415 and 0.00238, respectively. However, they are the only two of the thirty embedding–pooling configurations that outperform TF-IDF. The results therefore support these particular combinations rather than a general claim that dense pretrained representations are superior to sparse lexical features.

Max–supervised-attention pooling produces the highest score within all five embedding sources. Its scores for BPEmb, Word2Vec, and LexVec are 0.95612, 0.97147, and 0.96715, respectively. This consistent within-source ranking suggests that combining coordinate-wise maxima with a task-dependent weighted summary is useful under the evaluated settings. The two components may preserve complementary information: max pooling retains prominent values in individual embedding dimensions, while supervised attention learns how to weight tokens using the training labels.

Nevertheless, the results do not show that supervised attention alone is sufficient. For FastText, supervised attention obtains 0.94219, below mean pooling at 0.97245. The same pattern is particularly pronounced for BPEmb, where supervised attention obtains 0.88638 compared with 0.95366 for mean pooling. GloVe Twitter, in contrast, benefits from supervised attention relative to mean pooling. These differences indicate that the effectiveness of the learned weighting mechanism depends on the embedding resource and its interaction with the auxiliary training procedure.

The combined strategy also doubles the feature dimensionality relative to an individual pooling operator. Its advantage cannot therefore be attributed exclusively to attention. Moreover, in the combined configuration, the attention component is trained in the presence of max-pooled features; it is not simply the independently trained attention vector appended after training. Additional ablations would be needed to distinguish the effects of increased

**Table 8: RQ3: effect of the classification head on a fine-tuned `roberta-base` encoder. Mean ± standard deviation over the same five folds. Macro ROC-AUC is the primary metric. Both models share the encoder, input (`comment_heavy`, max. 128 tokens) and training setup (AdamW, lr $2 \times 10^{-5}$, 2 epochs, batch 16, BCE loss); only the head differs. Params are all trainable parameters, since the 124.6M encoder is fine-tuned rather than frozen. Best value per column in bold.**

| Family | Model | Params | **Macro ROC-AUC** | Micro ROC-AUC | Micro-F1 | Macro-F1 |
|---|---|---|---|---|---|---|
| Transformer | RoBERTa | 124.65M | **0.9887** ± 0.0014 | **0.9914** ± 0.0005 | 0.7742 ± 0.0038 | 0.6084 ± 0.0340 |
| Transformer | RoBERTa + LD | 124.95M | 0.9881 ± 0.0011 | 0.9911 ± 0.0008 | **0.7745** ± 0.0062 | **0.6189** ± 0.0206 |

**Table 9: RQ3: per-label results of RoBERTa with a linear head and with the label-dependency (LD) head. Mean ± standard deviation over the same five folds; F1 uses a fixed threshold of 0.5. Labels are ordered from most to least frequent; #Pos is the number of positive training comments. ΔF1 is the difference of the mean F1 (LD − RoBERTa). Better value per label and metric in bold.**

| | | F1 | | | ROC-AUC | |
|---|---|---|---|---|---|---|
| Label | #Pos | RoBERTa | RoBERTa + LD | ΔF1 | RoBERTa | RoBERTa + LD |
| *toxic* | 15,290 | 0.8139 ± 0.0090 | **0.8175** ± 0.0047 | +0.0036 | **0.9839** ± 0.0013 | 0.9833 ± 0.0009 |
| *obscene* | 8,446 | **0.8389** ± 0.0096 | 0.8381 ± 0.0100 | −0.0008 | **0.9940** ± 0.0005 | 0.9936 ± 0.0009 |
| *insult* | 7,874 | **0.7511** ± 0.0102 | 0.7474 ± 0.0211 | −0.0037 | **0.9878** ± 0.0009 | 0.9874 ± 0.0013 |
| *severe_toxic* | 1,594 | **0.4020** ± 0.0963 | 0.3831 ± 0.1082 | −0.0189 | **0.9905** ± 0.0011 | 0.9904 ± 0.0010 |
| *identity_hate* | 1,405 | **0.5179** ± 0.0366 | 0.5016 ± 0.0903 | −0.0163 | 0.9885 ± 0.0045 | **0.9887** ± 0.0017 |
| *threat* | 478 | 0.3268 ± 0.1786 | **0.4256** ± 0.0494 | +0.0988 | **0.9873** ± 0.0041 | 0.9853 ± 0.0041 |
| Macro average | | 0.6084 ± 0.0340 | **0.6189** ± 0.0206 | +0.0105 | **0.9887** ± 0.0014 | 0.9881 ± 0.0011 |

dimensionality, complementary information, and joint auxiliary training.

TF-IDF remains an important baseline because it preserves specific lexical patterns without compressing them into a dense document vector. Although the classifier family is held fixed in RQ1, the representations differ in dimensionality, pretraining data, token coverage, and use of supervision. The comparison thus evaluates practical representation pipelines rather than isolating the effect of any single embedding algorithm.

## 7.2 RQ2: Several Neural Architectures Achieve Similar Performance

With the embedding source fixed, the clearest effect separates pooled-feature classifiers from learned sequence encoders. Logistic regression and LightGBM on the pooled FastText vectors reach 0.9715 and 0.9800, whereas all four sequence models reach 0.9851–0.9877, and the best sequence models outperform both pooled-feature classifiers on every fold. A fixed pooling of frozen word vectors discards word order and local composition, and a non-linear classifier recovers only part of that loss.

Among the sequence models, the comparison shows closely matched Macro ROC-AUC scores for the gated recurrent models and DPCNN under the selected settings. These architectures process text differently: recurrent encoders model sequences through hidden-state updates, whereas DPCNN builds hierarchical features using convolution and downsampling. Their similar aggregate performance suggests that multiple architectural approaches can capture useful toxicity-related evidence with the selected pretrained embeddings.

The paired fold comparisons are useful for interpreting small differences in mean performance. A slightly higher average does not by itself establish a reliable architectural advantage, particularly when the differences vary across folds. Conversely, close observed scores do not establish statistical equivalence. The findings apply to the evaluated model sizes, sequence lengths, optimization settings, and training budget, rather than to all possible configurations of these architecture families.

The pooling mechanisms in RQ1 and RQ2 also serve different roles. In RQ1, pooling aggregates fixed pretrained token embeddings into document features for logistic regression. Supervised attention is learned through an auxiliary classification task before the downstream classifier is fitted. In the recurrent models, max pooling and learned attention aggregate contextual encoder states and are optimized jointly with the final classifier. Consequently, the recurrent results should not be interpreted as a direct replication of the RQ1 pooling comparison.

## 7.3 RQ3: Explicit Label Dependency Does Not Improve the Baseline

Standard RoBERTa achieves a mean Macro ROC-AUC of 0.9887, compared with 0.9881 for the label-dependency extension. The observed difference of 0.0006 is small, but it provides no evidence of an improvement from the added component under the evaluated configuration. The result illustrates that label co-occurrence alone does not guarantee that an explicit dependency mechanism will improve prediction.

One possible explanation is that the shared contextual encoder already learns features relevant to multiple toxicity labels, limiting

the additional benefit of the dependency head. Another possibility is that corpus-level conditional co-occurrence estimates do not adequately represent the relationships applicable to each individual comment. Estimates involving rare labels may also be sensitive to the limited number of positive training examples. Additional parameters and optimization requirements provide further possible explanations. The present experiment does not isolate these mechanisms, so they should be treated as hypotheses.

This finding is specific to the evaluated dependency construction, prediction head, and training procedure. It does not establish that label-dependency modeling is generally ineffective. Further ablations could compare fixed and learned dependency structures, vary the contribution of the dependency component, and repeat training with additional random seeds. Such experiments would help determine whether the negative result reflects the selected implementation or a limited benefit from explicit dependency modeling in this setting.

### 7.4 Evaluation Scope and Practical Implications

The evaluation protocols constrain how the results should be compared. RQ1 uses a single holdout partition, whereas RQ2 and RQ3 use five-fold evaluation. Accordingly, the scores should be interpreted primarily within each research question, not as a unified ranking of every model in the study. In particular, the observed RQ1 improvements over TF-IDF have not been shown to persist across multiple partitions or random seeds. Selecting the best configuration from many candidates on the same validation partition can also yield an optimistic estimate of its generalization performance. Independent evaluation would strengthen the evidence for the selected representation.

Macro ROC-AUC measures label-wise ranking performance without requiring a decision threshold. It does not directly establish the quality of binary moderation decisions at a particular operating point. A model with strong ROC-AUC can still have limited precision or recall for an infrequent label at a threshold of 0.5. Threshold-dependent metrics therefore provide complementary information. Any threshold selection should use training or validation data rather than the held-out evaluation labels.

Practical model selection must also account for training cost, inference latency, memory requirements, and maintenance. These costs have not been systematically benchmarked here, so the predictive results do not establish an efficiency ranking. The findings nevertheless support retaining simple baselines and requiring empirical justification for additional representation or architectural complexity.

## 8 Limitations and Ethical Considerations

**Limitations.** First, all models use a fixed threshold of 0.5, and all except logistic regression use no class weighting, so their F1 scores understate what they could reach with per-label thresholds or a weighted loss; the comparison on F1 is therefore partly a comparison of decision rules. Second, hyperparameters were not tuned: the neural models were trained for a fixed budget of 4 epochs, LightGBM used default-sized trees without class weights, and RoBERTa was fine-tuned for only 2 epochs. Third, heavy cleaning removes stopwords and punctuation, which suits bag-of-words models but discards context that a transformer could use. Fourth, the RoBERTa folds were built before the 75 comments that are empty after heavy cleaning were excluded, so they contain 159,476 instead of 159,401 comments; this affects less than 0.05% of the data. Finally, all results come from a single English dataset of Wikipedia comments, and toxicity labels are subjective, so performance on other platforms or languages may differ.

**Ethical considerations.** Toxicity classifiers can learn spurious associations between identity terms (e.g., names of religions or sexual orientations) and toxicity, and then flag non-toxic comments that merely mention these groups [6]. They can also be biased against dialects such as African American English [16]. False positives silence legitimate speech, while false negatives leave users exposed to abuse, and these errors are not spread evenly across groups. We have not measured such biases in this work. The models should therefore support, not replace, human moderators, and should be audited for bias before deployment. Attention weights of the recurrent models can help reviewers, but they are not a faithful explanation of a prediction [10]. Finally, the dataset itself contains offensive language, and the examples in this report are shown only to illustrate the task.

## 9 Conclusion and Future Work

This study investigated three design choices for multi-label toxic comment classification: text representation, neural architecture, and explicit label-dependency modeling. Macro ROC-AUC was used as the primary metric, with comparisons organized around these three research questions.

For RQ1, FastText with max–supervised-attention pooling achieved the highest holdout Macro ROC-AUC of 0.97956, followed by GloVe Twitter with the same pooling strategy at 0.97779. Both exceeded the TF-IDF baseline of 0.97541. Although max–supervised-attention pooling performed best within all five embedding sources, only these two embedding–pooling configurations outperformed TF-IDF. Supervised attention alone did not consistently improve performance. The findings therefore favor evaluating embedding resources and pooling strategies jointly, while retaining sparse lexical features as a strong reference baseline.

For RQ2, learned sequence encoders outperformed logistic regression and LightGBM on pooled FastText features on every fold (0.9851–0.9877 versus 0.9715 and 0.9800), and the gated recurrent encoders and DPCNN obtained closely matched scores under the selected embedding source and training settings. Most of the differences came from the rarest labels, above all *threat*. These results do not establish a universally preferable neural architecture. For RQ3, the explicit label-dependency extension did not improve the matched RoBERTa baseline: mean Macro ROC-AUC was 0.9881 for the extended model and 0.9887 for standard RoBERTa. This result emphasizes the need to validate additional model components rather than assuming that greater complexity produces better predictions.

The project also provides a modular Python implementation with reusable experiment components, persisted models and metrics, and interchangeable in-memory and CSV dataset access in the repository-based workflow. This separation supports maintainable

experimentation without coupling the core processing logic to a single dataset storage backend.

Future work should validate the representation results across multiple folds and random seeds, assess selected configurations on independent data, and benchmark computational costs alongside predictive performance. Further label-dependency ablations and comparisons between text-preprocessing strategies would help clarify the scope of the findings. Overall, the study supports selecting representations and model components through matched empirical comparisons, with conclusions limited to the evaluation settings that support them.

## References


[1] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. 1994. Learning Long-Term Dependencies with Gradient Descent is Difficult. *IEEE Transactions on Neural Networks* 5, 2 (1994), 157–166.

[2] Zhao-Min Chen, Xiu-Shen Wei, Peng Wang, and Yanwen Guo. 2019. Multi-Label Image Recognition with Graph Convolutional Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 5177–5186.

[3] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 1724–1734.

[4] Thomas Davidson, Dana Warmsley, Michael Macy, and Ingmar Weber. 2017. Automated Hate Speech Detection and the Problem of Offensive Language. In *Proceedings of the International AAAI Conference on Web and Social Media (ICWSM)*. 512–515.

[5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. 4171–4186.

[6] Lucas Dixon, John Li, Jeffrey Sorensen, Nithum Thain, and Lucy Vasserman. 2018. Measuring and Mitigating Unintended Bias in Text Classification. In *Proceedings of the 2018 AAAI/ACM Conference on AI, Ethics, and Society (AIES)*. 67–73.

[7] Paula Fortuna and Sérgio Nunes. 2018. A Survey on Automatic Detection of Hate Speech in Text. *Comput. Surveys* 51, 4 (2018), 85:1–85:30.

[8] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*. 1024–1034.

[9] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Identity Mappings in Deep Residual Networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*. 630–645.

[10] Sarthak Jain and Byron C. Wallace. 2019. Attention is not Explanation. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. 3543–3556.

[11] Rie Johnson and Tong Zhang. 2017. Deep Pyramid Convolutional Neural Networks for Text Categorization. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*. 562–570.

[12] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems (NeurIPS)*. 3146–3154.

[13] Yoon Kim. 2014. Convolutional Neural Networks for Sentence Classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 1746–1751.

[14] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692* (2019).

[15] Tomas Mikolov, Edouard Grave, Piotr Bojanowski, Christian Puhrsch, and Armand Joulin. 2018. Advances in Pre-Training Distributed Word Representations. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC)*.

[16] Maarten Sap, Dallas Card, Saadia Gabriel, Yejin Choi, and Noah A. Smith. 2019. The Risk of Racial Bias in Hate Speech Detection. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*. 1668–1678.

[17] Anna Schmidt and Michael Wiegand. 2017. A Survey on Hate Speech Detection using Natural Language Processing. In *Proceedings of the Fifth International Workshop on Natural Language Processing for Social Media*. 1–10.

[18] Mike Schuster and Kuldip K. Paliwal. 1997. Bidirectional Recurrent Neural Networks. *IEEE Transactions on Signal Processing* 45, 11 (1997), 2673–2681.

[19] Betty van Aken, Julian Risch, Ralf Krestel, and Alexander Löser. 2018. Challenges for Toxic Comment Classification: An In-Depth Error Analysis. In *Proceedings of the 2nd Workshop on Abusive Language Online (ALW2)*. 33–42.

[20] Sida Wang and Christopher D. Manning. 2012. Baselines and Bigrams: Simple, Good Sentiment and Topic Classification. In *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics (ACL)*. 90–94.

[21] Ellery Wulczyn, Nithum Thain, and Lucas Dixon. 2017. Ex Machina: Personal Attacks Seen at Scale. In *Proceedings of the 26th International Conference on World Wide Web (WWW)*. 1391–1399.

[22] Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard Hovy. 2016. Hierarchical Attention Networks for Document Classification. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. 1480–1489.

[23] Min-Ling Zhang and Zhi-Hua Zhou. 2014. A Review on Multi-Label Learning Algorithms. *IEEE Transactions on Knowledge and Data Engineering* 26, 8 (2014), 1819–1837.